MODULE 03 · DOMAIN 1 - DESIGN APPLICATIONS

Prompts that return a format

Labels, JSON and nothing else. Then when to stop asking and start enforcing.

SECTION 1 of 6 EXAM WEIGHT 14% MODULE 03 of 40

admin@sthithapragna.com

Chat models are trained to be helpful, and helpful means chatty. **Write the prompt as if you are talking to a form, not a friend.**

The six parts of a formatting prompt

PART	WHAT IT DOES
Task instruction	Says what to do, in one plain sentence
Delimited input	Marks where the input starts and ends, so it cannot be confused with instructions
Exact output spec	Lists allowed labels, or the keys and types of the JSON
Example	Shows one finished output in the exact form wanted
Negative constraint	No explanation, no preamble, no code fences
Missing-value rule	What to return when the answer is not in the input

ON THE EXAM

When the exam offers several prompts, the most reliable one usually contains the most of these six. Count them.

admin@sthithapragna.com

A label prompt, weak and strong

WEAK

Is this review positive?

- No list of allowed answers
- No boundary around the review
- Invites a sentence in reply
- Nothing said about uncertainty

STRONG

Answer with exactly one word

- Allowed: POSITIVE, NEGATIVE, MIXED
- Review sits between triple backticks
- Output only the word. No explanation
- If unclear, output MIXED

THE TIEBREAKER

Name every allowed string exactly as it must appear, including case. A model cannot match a list it was never given.

admin@sthithapragna.com

What a reliable JSON prompt contains

- The **exact keys**, their types, and whether each may be null.
- One **example object** in exactly the shape wanted.
- Return only the JSON object. No prose before or after, no code fences.
- Use null for anything not stated in the input. Never guess.
- Keep the structure **flat and simple**. Every extra level is another place to go wrong.

> ON THE EXAM
>
> An example object teaches the format more reliably than a paragraph describing it.

Asking for a format, or enforcing one?

PROMPT INSTRUCTIONS

Asking

- Works on any model
- Very reliable when written well
- Still probabilistic
- Needs validation downstream

STRUCTURED OUTPUTS

Enforcing

- A JSON schema sent with the request
- Output constrained to match it
- Supported on Foundation Model API chat models
- Only a subset of JSON Schema allowed

THE TIEBREAKER

If an option enforces the schema and the others only ask nicely, the enforcing option is the most reliable. If all options are prompts, pick the most complete prompt.

admin@sthithapragna.com

Structured outputs: what to know

FACT	DETAIL
How	response_format with type json_schema, and strict set to true
Where	Chat models on pay-per-token and provisioned throughput endpoints
Not supported	pattern, anyOf, oneOf, allOf, prefixItems, and $ref
Limit	Up to 64 keys. Lengths of arrays and strings are not enforced
Recommended for	Extraction from many documents, and batch jobs needing a fixed format

ON THE EXAM

Databricks deliberately supports a simpler subset because simpler schemas generate higher-quality JSON. Design to the subset.

admin@sthithapragna.com

Examples are the strongest format instruction

- **Zero-shot** describes the format. **Few-shot** shows it, and usually wins.
- Make every example identical in shape. Inconsistency teaches inconsistency.
- Include one awkward case, such as a missing value, answered correctly.
- Two or three examples are usually enough for format.
- Examples with explanations teach the model to explain.

> ON THE EXAM
>
> The model copies what the examples do, not what the instructions say. Make them agree.

admin@sthithapragna.com

The output is right, but it keeps explaining itself

- State it plainly: output only the answer, with no explanation of how it was produced.
- Show examples that contain **only the answer**.
- Put persistent rules in the **system prompt**, which applies to every turn.
- Use structured output, so there is no free-text field to explain in.
- Stripping text afterwards with code treats the symptom. Keep it as a safety net only.

> ON THE EXAM
>
> Fix it in the prompt or the schema first. Post-processing is a backstop, not the design.

admin@sthithapragna.com

One job, several prompts, each output feeding the next

01
Extract
Pull the facts out as JSON.

›

02
Classify
Pick a category from the facts.

›

03
Draft
Write the reply using facts and category.

›

04
Check
Verify the draft against the facts.

ON THE EXAM

Each link has one job and one output format, so each can be tested, and the failing link can be found.

One prompt or a chain?

ONE PROMPT IS FINE WHEN

The job is a single step

- One task, one output
- Latency budget is tight
- Output is easy to check
- Failures are rare

CHAIN WHEN

The job is several steps

- Several tasks in one request
- Outputs are inconsistent
- You cannot tell which part fails
- Steps suit different models

THE TIEBREAKER

A chain costs more calls and more latency. It buys reliability and debuggability. Choose it when those are the problem.

admin@sthithapragna.com

Harbourline's claims inbox

01
Email arrives
Free text from a policyholder.

02
Extract
Structured output: policy number, date, loss type.

03
Classify
One label: NEW CLAIM, UPDATE or COMPLAINT.

04
Route
Label decides the queue.

05
Validate
Nulls flagged for a person to check.

ON THE EXAM

Structured output where a program reads the result. A labelled, one-word answer where a router reads it.

An airline tags passenger feedback with one of four categories: DELAY, BAGGAGE, STAFF or OTHER. A SQL job reads the tag directly.

The model picks the right category, but often replies with text such as: The category is Delay because the flight left late.

Which change will most reliably fix the output?

A The four exact values, only one in capitals, no other text, with examples

B Ask the model to explain its reasoning first, then give the category

C Raise the temperature so the model answers more decisively each time

D Tell the model it is an expert airline customer feedback analyst

◆ PAUSE AND DECIDE

Which change will most reliably fix the output?

A Exact values, only one, examples

Adds the output spec, the negative constraint and an example.

B Reasoning first, then category

Guarantees more text in the output, the opposite of what is needed.

C Higher temperature

Adds randomness. Format reliability goes down, not up.

D Expert persona

May improve tone. Says nothing about format.

THE SIGNAL

The category is right; the wrapping is wrong. That is a format problem, so fix the output specification.

A batch job sends 500,000 invoices through a Foundation Model API chat model to extract supplier, total and due date as JSON. The prompt already lists the keys and includes an example.

About two percent of responses still fail to parse, because of code fences, trailing comments or a missing key.

What should the engineer do to make the output reliably parseable?

A Rewrite the instructions in capitals and repeat them twice

B Strip code fences and trailing text with a regular expression after each call

C Send a JSON schema in response_format with strict structured outputs

D Raise max tokens so the model has room to finish the object

◆ PAUSE AND DECIDE

What should the engineer do to make the output reliably parseable?

A **Capitals, repeated**

Still only asking. The prompt was already good.

B **Regex clean-up afterwards**

Fixes fences, but cannot restore a missing key. Treats a symptom.

C **Strict structured outputs**

Enforces the schema at generation. Built for batch extraction.

D **Raise max tokens**

Nothing suggests truncation. Fences and comments are not length faults.

THE SIGNAL

The prompt already lists keys and gives an example. When asking well is not enough, move from asking to enforcing.

A single prompt reads a long customer complaint, extracts the key facts, decides a liability category, and drafts a reply in the brand's tone.

Results vary from run to run, and when a reply is wrong the team cannot tell whether the facts, the category or the drafting failed.

What should the engineer change?

A Move the same single prompt to a larger, more capable model

B Add more detailed instructions to the existing single prompt

C Fine-tune a model on the team's past complaint replies instead

D Split the work into a chain: extract, then classify, then draft

◆ PAUSE AND DECIDE

What should the engineer change?

A **Larger model**

Might help a little, but you still cannot see which part failed.

B **More instructions**

Makes the overloaded prompt longer and more fragile.

C **Fine-tune on past replies**

Learns tone, not correct facts or categories. Costly, and still opaque.

D **Chain of three prompts**

Each step is simpler, testable, and shows where failures occur.

THE SIGNAL

Several tasks in one prompt, inconsistent results, and no way to find the failing part. All three are chaining signals.

> An engineer defines a strict JSON schema for addresses. It uses anyOf to allow either a UK or a US address shape, and a regex pattern to validate postcodes.
>
> Requests using this schema in response_format fail on a Foundation Model API endpoint.

What is the best way forward?

A Drop structured outputs and rely on prompt instructions alone instead

B One address object with nullable fields, and validate postcodes afterwards

C Increase max tokens so that the schema has room to be applied fully

D Switch to a larger model that understands complex schemas better

◆ PAUSE AND DECIDE

What is the best way forward?

A Prompt instructions only

Gives up the guarantee to avoid a fixable schema problem.

B Simpler schema, validate afterwards

Stays inside the supported subset and keeps enforcement.

C More max tokens

Token limits have nothing to do with unsupported keywords.

D Larger model

The limit is in the API's supported schema subset, not the model.

THE SIGNAL

anyOf and pattern are outside the supported subset. Design the schema to the subset and move rich validation into code.

What to carry forward

- Six parts: **instruction, delimited input, exact spec, example, negative constraint, missing-value rule**.
- Examples beat descriptions. Examples with explanations teach explaining.
- Asking versus enforcing: **strict structured outputs** win when a program reads the result.
- Design schemas to the supported subset. No anyOf, no pattern, 64 keys at most.
- Chain when one prompt does several jobs and you cannot see which one fails.

> ON THE EXAM
>
> Next: module 04 gives the model tools, and decides who chooses which one to call.